

Competitive Exam Practice 5: Queues, Circular Buffers, Deques, and Priority Queues

GATE (Computer Science) Pattern

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 17, 2026

Each question carries 1–2 marks. MCQ = single correct option; NAT = Numerical Answer Type.

Q1 [*GATE CSE 1996 Q1.12, verified — GATEOverflow V2 p.240*] [**MCQ, 1 mark**] Consider the following statements:

- (i) First-in-first out types of computations are efficiently supported by STACKS.
- (ii) Implementing LISTS on linked lists is more efficient than implementing LISTS on an array for almost all the basic LIST operations.
- (iii) Implementing QUEUES on a circular array is more efficient than implementing QUEUES on a linear array with two indices.
- (iv) Last-in-first-out type of computations are efficiently supported by QUEUES.

Which of the following is correct?

- (A) (ii) and (iii) are true
- (B) (i) and (ii) are true
- (C) (iii) and (iv) are true
- (D) (ii) and (iv) are true

Q2 [*GATE CSE 2012 Q35, verified — GATEOverflow V2 p.241*] [**MCQ, 2 marks**] Suppose a circular queue of capacity $(n - 1)$ elements is implemented with an array of n elements. Assume that the insertion and deletion operations are carried out using REAR and FRONT as array index variables, respectively. Initially, REAR = FRONT = 0. The conditions to detect queue full and queue empty are:

- (A) full: $(\text{REAR}+1) \bmod n == \text{FRONT}$ empty: $\text{REAR} == \text{FRONT}$
- (B) full: $(\text{REAR}+1) \bmod n == \text{FRONT}$ empty: $(\text{FRONT}+1) \bmod n == \text{REAR}$
- (C) full: $\text{REAR} == \text{FRONT}$ empty: $(\text{REAR}+1) \bmod n == \text{FRONT}$
- (D) full: $(\text{FRONT}+1) \bmod n == \text{REAR}$ empty: $\text{REAR} == \text{FRONT}$

Q3 [*GATE CSE 2016 Set 1 Q10, verified — GATEOverflow V2 p.242*] [**MCQ, 1 mark**] A queue is implemented using an array such that ENQUEUE and DEQUEUE operations are performed efficiently. Which one of the following statements is CORRECT (n refers to the number of items in the queue)?

- (A) Both operations can be performed in $O(1)$ time
- (B) At most one operation can be performed in $O(1)$ time but the worst case time for the other operation will be $\Omega(n)$
- (C) The worst case time complexity for both operations will be $\Omega(n)$
- (D) Worst case time complexity for both operations will be $\Omega(\log n)$

Q4 [GATE CSE 1997 Q4.7, verified — GATEOverflow V2 p.239] [MCQ, 1 mark] A priority queue Q is used to implement a stack that stores characters. $PUSH(C)$ is implemented as $INSERT(Q, C, K)$ where K is an appropriate integer key chosen by the implementation. POP is implemented as $DELETEMIN(Q)$. For a sequence of operations, the keys chosen are in

- (A) non-increasing order
- (B) non-decreasing order
- (C) strictly increasing order
- (D) strictly decreasing order

Q5 [GATE IT 2007 Q30, verified — GATEOverflow V2 pp.243–244] [MCQ, 2 marks] Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:

- (i) $isEmpty(Q)$ — returns true if the queue is empty, false otherwise.
- (ii) $delete(Q)$ — deletes the element at the front of the queue and returns its value.
- (iii) $insert(Q, i)$ — inserts the integer i at the rear of the queue.

Consider the following function:

```
void f (queue Q) {
    int i;
    if (!isEmpty(Q)) {
        i = delete(Q);
        f(Q);
        insert(Q, i);
    }
}
```

What operation is performed by the above function f ?

- (A) Leaves the queue Q unchanged
- (B) Reverses the order of the elements in the queue Q
- (C) Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order
- (D) Empties the queue Q

Q6 [Original, GATE-pattern] [MCQ, 2 marks] A circular queue of capacity 6 uses the count version ($count = 0$ means empty, $count = 6$ means full), with indices advancing modulo 6. At some moment $front = 4$, $rear = 2$, $count = 5$. One $dequeue$ followed by one $enqueue$ is performed. Afterwards, $(front, rear, count)$ equals:

- (A) (5, 3, 5)
- (B) (5, 3, 6)
- (C) (4, 3, 5)
- (D) (5, 2, 5)

Q7 [*Original, GATE-pattern*] [MCQ, 1 mark] A double-ended queue supports insertions at *both* ends but deletions from the *front* only. Which restricted variant is this, and which operation is forbidden?

- (A) Input-restricted deque; `enqueueFront` is forbidden
- (B) Output-restricted deque; `dequeueRear` is forbidden
- (C) Input-restricted deque; `dequeueRear` is forbidden
- (D) Output-restricted deque; `enqueueFront` is forbidden

Q8 [*Original, GATE-pattern*] [MCQ, 2 marks] A priority queue holds up to $n = 10,000$ items in an array (no heap allowed). The workload is 10,000 inserts followed by 5 `deleteMin` operations. Which design gives the smaller worst-case total time?

- (A) Unordered array: 10,000 $O(1)$ inserts plus 5 $O(n)$ scans
- (B) Ordered array: 10,000 $O(n)$ inserts plus 5 $O(1)$ removals
- (C) Both designs cost the same asymptotically on this workload
- (D) Ordered array, because `deleteMin` dominates the insert cost